

```
"""
```

```
Lab 10 – E-Commerce Sales Analysis Script  
Data Analysis with Python · GUC CCE
```

```
Converts the Lab 9 notebook analysis into a standalone Python script.
```

```
Usage:
```

```
python ecommerce_analysis.py  
python3 ecommerce_analysis.py
```

```
Output:
```

```
output/ecommerce_eda_summary.csv  
output/ecommerce_distributions.png  
output/ecommerce_correlation.png  
output/ecommerce_profit_by_category.png  
output/ecommerce_timeseries.png  
output/ecommerce_regression_results.csv  
output/ecommerce_confusion_matrix.png  
output/ecommerce_roc_curve.png
```

```
"""
```

```
import os  
import pandas as pd  
import numpy as np  
import matplotlib  
matplotlib.use("Agg")  
import matplotlib.pyplot as plt  
import seaborn as sns  
from sklearn.linear_model import LinearRegression, LogisticRegression  
from sklearn.model_selection import train_test_split  
from sklearn.metrics import (r2_score, mean_squared_error,  
                             accuracy_score, classification_report,  
                             confusion_matrix, roc_curve, roc_auc_score,  
                             f1_score)  
  
import warnings  
warnings.filterwarnings('ignore')  
  
sns.set_theme(style="whitegrid", font_scale=1.2)  
  
OUTPUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "output")  
os.makedirs(OUTPUT_DIR, exist_ok=True)  
  
_SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))  
_CANDIDATES = [
```

```

        os.path.join(_SCRIPT_DIR, "Ecommerce_Sales_Analysis.csv"),
        os.path.join(_SCRIPT_DIR, "..", "..", "Assignments", "Ecommerce_Sales_Analysi
]
DATA_FILE = next((p for p in _CANDIDATES if os.path.exists(p)), _CANDIDATES[0])

# -----
# Dark theme helper
# -----
BG_DARK = '#0f1117'
BG_PANEL = '#1a1d2e'
C_TEXT = '#c9d1d9'
C_TITLE = '#e6edf3'
C_BLUE = '#58a6ff'
C_GREEN = '#3fb950'
C_RED = '#f85149'
C_ORANGE = '#f0883e'
C_PURPLE = '#d2a8ff'
C_BORDER = '#30363d'

def _style_ax(ax):
    ax.set_facecolor(BG_PANEL)
    ax.tick_params(colors=C_TEXT)
    ax.xaxis.label.set_color(C_TEXT)
    ax.yaxis.label.set_color(C_TEXT)
    ax.title.set_color(C_TITLE)
    for spine in ax.spines.values():
        spine.set_edgecolor(C_BORDER)

# -----
def load_and_prepare():
    df = pd.read_csv(DATA_FILE)
    df['Order Date'] = pd.to_datetime(df['Order Date'], format='%m/%d/%Y')
    df['Ship Date'] = pd.to_datetime(df['Ship Date'], format='%m/%d/%Y')
    df['is_profitable'] = (df['Profit'] > 0).astype(int)
    df['Profit_Margin'] = df['Profit'] / df['Sales'] * 100
    print(f"Loaded E-Commerce: {df.shape[0]:,} rows, {df.shape[1]} columns")
    print(f"Profitable orders: {df['is_profitable'].mean():.1%}")
    return df

def run_eda(df):
    print("\n=== Descriptive Statistics ===")
    summary = df[["Sales", "Quantity", "Discount", "Profit", "Profit_Margin"]].de
    print(summary)
    summary.to_csv(os.path.join(OUTPUT_DIR, "ecommerce_eda_summary.csv"))

```

```

# — Chart 1: Distributions —————
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.patch.set_facecolor(BG_DARK)
for ax in axes.flat:
    _style_ax(ax)

axes[0,0].hist(df['Sales'].clip(upper=2000), bins=60, color=C_BLUE, edgecolor=
axes[0,0].set_title('Sales Distribution', fontweight='bold')
axes[0,0].set_xlabel('Sales ($)')
axes[0,0].set_ylabel('Count')
m = df['Sales'].median()
axes[0,0].axvline(m, color=C_ORANGE, lw=2, linestyle='--', label=f'Median: ${
axes[0,0].legend(facecolor=BG_PANEL, labelcolor=C_TITLE)

axes[0,1].hist(df['Profit'], bins=80, color=C_GREEN, edgecolor='none', alpha=
axes[0,1].axvline(0, color=C_RED, lw=2, linestyle='--', label='Break-even')
axes[0,1].set_title('Profit Distribution', fontweight='bold')
axes[0,1].set_xlabel('Profit ($)')
axes[0,1].set_ylabel('Count')
axes[0,1].legend(facecolor=BG_PANEL, labelcolor=C_TITLE)

sc = axes[1,0].scatter(df['Discount'], df['Profit'],
                        alpha=0.25, c=df['Sales'], cmap='plasma', s=10)
axes[1,0].axhline(0, color=C_RED, lw=1.5, linestyle='--')
axes[1,0].set_title('Profit vs Discount', fontweight='bold')
axes[1,0].set_xlabel('Discount Rate')
axes[1,0].set_ylabel('Profit ($)')
cbar = plt.colorbar(sc, ax=axes[1,0])
cbar.set_label('Sales ($)', color=C_TEXT)
cbar.ax.yaxis.set_tick_params(color=C_TEXT)
plt.setp(plt.getp(cbar.ax.axes, 'yticklabels'), color=C_TEXT)

axes[1,1].hist(df['Quantity'], bins=14, color=C_PURPLE, edgecolor='none', alp
axes[1,1].set_title('Quantity Distribution', fontweight='bold')
axes[1,1].set_xlabel('Quantity Ordered')
axes[1,1].set_ylabel('Count')
avg_q = df['Quantity'].mean()
axes[1,1].axvline(avg_q, color=C_ORANGE, lw=2, linestyle='--', label=f'Mean:
axes[1,1].legend(facecolor=BG_PANEL, labelcolor=C_TITLE)

fig.suptitle('E-Commerce Sales – Exploratory Data Analysis',
             fontsize=16, fontweight='bold', color=C_TITLE, y=1.01)
plt.tight_layout()
fig.savefig(os.path.join(OUTPUT_DIR, "ecommerce_distributions.png"),
            dpi=150, bbox_inches='tight', facecolor=BG_DARK)
plt.close(fig)
print("  Saved: ecommerce_distributions.png")

```

```

# — Chart 2: Category / Segment —————
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.patch.set_facecolor(BG_DARK)
for ax in axes: _style_ax(ax)

cats = df.groupby('Category')[['Sales', 'Profit']].sum().sort_values('Sales',
x = np.arange(len(cats)); w = 0.35
b1 = axes[0].bar(x-w/2, cats['Sales']/1000, w, color=C_BLUE, label='Sales',
b2 = axes[0].bar(x+w/2, cats['Profit']/1000, w, color=C_GREEN, label='Profit'
axes[0].set_xticks(x)
axes[0].set_xticklabels(cats.index, rotation=0, fontsize=10)
axes[0].set_title('Sales & Profit by Category', fontweight='bold')
axes[0].set_ylabel('Amount ($K)')
axes[0].legend(facecolor=BG_PANEL, labelcolor=C_TITLE)
for bar in b1: axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0
    f'${bar.get_height():.0f}K', ha='center', va='bottom', fontsize=8, color=
for bar in b2: axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0
    f'${bar.get_height():.0f}K', ha='center', va='bottom', fontsize=8, color=

seg = df.groupby('Segment')['Profit'].sum().sort_values()
cols_seg = [C_RED, C_ORANGE, C_GREEN]
bars = axes[1].barh(seg.index, seg.values/1000, color=cols_seg, alpha=0.9)
axes[1].set_title('Total Profit by Segment', fontweight='bold')
axes[1].set_xlabel('Profit ($K)')
for bar, v in zip(bars, seg.values):
    axes[1].text(v/1000+0.5, bar.get_y()+bar.get_height()/2,
        f'${v/1000:.1f}K', va='center', color=C_TITLE, fontsize=10)

sc_p = df.groupby('Sub-Category')['Profit'].sum().sort_values(ascending=False)
cols_sc = [C_GREEN if v > 0 else C_RED for v in sc_p.values]
axes[2].barh(sc_p.index[::-1], sc_p.values[::-1]/1000, color=cols_sc[::-1], a
axes[2].set_title('Top 10 Sub-Categories by Profit', fontweight='bold')
axes[2].set_xlabel('Total Profit ($K)')

fig.suptitle('Category & Segment Performance', fontsize=15, fontweight='bold'
plt.tight_layout()
fig.savefig(os.path.join(OUTPUT_DIR, "ecommerce_profit_by_category.png"),
            dpi=150, bbox_inches='tight', facecolor=BG_DARK)
plt.close(fig)
print(" Saved: ecommerce_profit_by_category.png")

# — Chart 3: Correlation + Boxplot —————
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.patch.set_facecolor(BG_DARK)
for ax in axes: _style_ax(ax)

```

```

corr = df[['Sales', 'Quantity', 'Discount', 'Profit', 'Profit_Margin']].corr().round(2)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='RdYlGn', center=0,
            vmin=-1, vmax=1, linewidths=0.5, ax=axes[0],
            annot_kws={'size':12, 'color':'black'},
            cbar_kws={'shrink': 0.8})
axes[0].set_title('Correlation Matrix', fontweight='bold', color=C_TITLE)
axes[0].tick_params(colors=C_TEXT)

region_data = [df[df['Region']==r]['Profit'].values for r in ['West', 'East', 'Central', 'South']]
bp = axes[1].boxplot(region_data, patch_artist=True,
                    labels=['West', 'East', 'Central', 'South'],
                    medianprops=dict(color=C_ORANGE, linewidth=2))
for patch, color in zip(bp['boxes'], [C_BLUE, C_GREEN, C_PURPLE, C_RED]):
    patch.set_facecolor(color); patch.set_alpha(0.7)
axes[1].axhline(0, color=C_RED, lw=1.5, linestyle='--', alpha=0.7)
axes[1].set_title('Profit Distribution by Region', fontweight='bold', color=C_TITLE)
axes[1].set_ylabel('Profit ($)')

fig.suptitle('Correlation & Regional Analysis', fontsize=15, fontweight='bold')
plt.tight_layout()
fig.savefig(os.path.join(OUTPUT_DIR, "ecommerce_correlation.png"),
            dpi=150, bbox_inches='tight', facecolor=BG_DARK)
plt.close(fig)
print(" Saved: ecommerce_correlation.png")

# — Chart 4: Time Series —————
monthly = df.groupby(df['Order Date'].dt.to_period('M')).agg(
    Sales=('Sales', 'sum'), Profit=('Profit', 'sum')
).reset_index()
monthly['Order Date'] = monthly['Order Date'].dt.to_timestamp()

fig, axes = plt.subplots(2, 1, figsize=(16, 10), sharex=True)
fig.patch.set_facecolor(BG_DARK)
for ax in axes: _style_ax(ax)

axes[0].fill_between(monthly['Order Date'], monthly['Sales']/1000, alpha=0.3,
                    color=C_BLUE, lw=2.5, marker='o', ms=4)
axes[0].plot(monthly['Order Date'], monthly['Sales']/1000,
             color=C_BLUE, lw=2.5, marker='o', ms=4)
axes[0].set_title('Monthly Sales Trend (2011-2014)', fontweight='bold')
axes[0].set_ylabel('Sales ($K)')

axes[1].fill_between(monthly['Order Date'], monthly['Profit']/1000,
                    where=monthly['Profit']>=0, alpha=0.3, color=C_GREEN, lw=2.5, marker='o', ms=4)
axes[1].fill_between(monthly['Order Date'], monthly['Profit']/1000,
                    where=monthly['Profit']<0, alpha=0.3, color=C_RED, lw=2.5, marker='o', ms=4)
axes[1].plot(monthly['Order Date'], monthly['Profit']/1000,
             color=C_GREEN, lw=2.5, marker='o', ms=4)

```

```

axes[1].axhline(0, color=C_RED, lw=1.5, linestyle='--')
axes[1].set_title('Monthly Profit Trend', fontweight='bold')
axes[1].set_ylabel('Profit ($K)')
axes[1].tick_params(axis='x', rotation=30)

fig.suptitle('Time Series Analysis - Sales & Profit', fontsize=15, fontweight
plt.tight_layout()
fig.savefig(os.path.join(OUTPUT_DIR, "ecommerce_timeseries.png"),
            dpi=150, bbox_inches='tight', facecolor=BG_DARK)
plt.close(fig)
print("  Saved: ecommerce_timeseries.png")

def run_models(df):
    cat_cols = ['Category', 'Segment', 'Region']
    df_enc = pd.get_dummies(df, columns=cat_cols, drop_first=True)

    features = ['Sales', 'Quantity', 'Discount',
                'Category_Office Supplies', 'Category_Technology',
                'Segment_Corporate', 'Segment_Home Office',
                'Region_East', 'Region_South', 'Region_West']
    X      = df_enc[features]
    y_reg  = df_enc['Profit']
    y_cls  = df_enc['is_profitable']

    X_temp, X_test, y_reg_temp, y_reg_test = train_test_split(X, y_reg, test_size
X_train, X_val, y_reg_train, y_reg_val = train_test_split(X_temp, y_reg_temp,

y_cls_train = y_cls.loc[X_train.index]
y_cls_val   = y_cls.loc[X_val.index]
y_cls_test  = y_cls.loc[X_test.index]

print(f"\nSplit: Train={len(X_train)}, Val={len(X_val)}, Test={len(X_test)}")

# — Linear Regression —————
print("\n=== Linear Regression: Predict Profit ===")
model_lr = LinearRegression().fit(X_train, y_reg_train)
results  = []
for name, Xs, ys in [("Train", X_train, y_reg_train),
                    ("Val",   X_val,   y_reg_val),
                    ("Test",  X_test,  y_reg_test)]:
    pred = model_lr.predict(Xs)
    r2    = r2_score(ys, pred)
    rmse  = np.sqrt(mean_squared_error(ys, pred))
    print(f" {name:5s}  R² = {r2:.4f}  RMSE = ${rmse:.2f}")
    results.append({"Set": name, "Model": "LinearReg", "Metric": "R²",   "Val
results.append({"Set": name, "Model": "LinearReg", "Metric": "RMSE", "Val

```

```

print("\n Top coefficients:")
coefs = pd.Series(model_lr.coef_, index=features)
for feat in coefs.abs().nlargest(5).index:
    print(f"    {feat:35s}: {coefs[feat]:+.3f}")

# — Logistic Regression —————
print("\n=== Logistic Regression: Predict is_profitable ===")
model_log = LogisticRegression(max_iter=1000, random_state=42).fit(X_train, y_train)
for name, Xs, ys in [("Train", X_train, y_cls_train),
                    ("Val", X_val, y_cls_val),
                    ("Test", X_test, y_cls_test)]:
    pred = model_log.predict(Xs)
    acc = accuracy_score(ys, pred)
    f1 = f1_score(ys, pred)
    print(f" {name:5s} Accuracy = {acc:.3f} F1 = {f1:.3f}")

y_pred_test = model_log.predict(X_test)
y_proba_test = model_log.predict_proba(X_test)[:, 1]
auc = roc_auc_score(y_cls_test, y_proba_test)

print(f"\n Test AUC: {auc:.4f}")
print()
print(classification_report(y_cls_test, y_pred_test,
                           target_names=["Not Profitable", "Profitable"]))

results.append({"Set": "Test", "Model": "LogisticReg", "Metric": "Accuracy",
               "Value": round(accuracy_score(y_cls_test, y_pred_test), 4)})
results.append({"Set": "Test", "Model": "LogisticReg", "Metric": "AUC", "Value": round(auc, 4)})
results.append({"Set": "Test", "Model": "LogisticReg", "Metric": "F1", "Value": round(f1_score(y_cls_test, y_pred_test), 4)})

pd.DataFrame(results).to_csv(
    os.path.join(OUTPUT_DIR, "ecommerce_regression_results.csv"), index=False)
print(" Saved: ecommerce_regression_results.csv")

# — Chart 5: Confusion Matrix + ROC —————
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.patch.set_facecolor(BG_DARK)
for ax in axes: _style_ax(ax)

cm = confusion_matrix(y_cls_test, y_pred_test)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Not Profitable', 'Profitable'],
            yticklabels=['Not Profitable', 'Profitable'],
            ax=axes[0], linewidths=0.5,
            annot_kws={'size': 14, 'weight': 'bold'})

```

```

axes[0].set_title('Confusion Matrix - Test Set', fontweight='bold')
axes[0].set_ylabel('Actual')
axes[0].set_xlabel('Predicted')

fpr, tpr, _ = roc_curve(y_cls_test, y_proba_test)
axes[1].plot(fpr, tpr, color=C_BLUE, lw=2.5,
             label=f'Logistic Regression (AUC = {auc:.3f})')
axes[1].plot([0,1],[0,1], '--', color='#6e7681', lw=1.5, label='Random (AUC =
axes[1].fill_between(fpr, tpr, alpha=0.15, color=C_BLUE)
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].set_title('ROC Curve - Test Set', fontweight='bold')
axes[1].legend(facecolor=BG_PANEL, labelcolor=C_TITLE, fontsize=10)

fig.suptitle('Machine Learning Results - Classification',
            fontsize=15, fontweight='bold', color=C_TITLE)
plt.tight_layout()
fig.savefig(os.path.join(OUTPUT_DIR, "ecommerce_confusion_matrix.png"),
            dpi=150, bbox_inches='tight', facecolor=BG_DARK)
plt.close(fig)
print("  Saved: ecommerce_confusion_matrix.png")

fig2, ax2 = plt.subplots(figsize=(8, 6))
fig2.patch.set_facecolor(BG_DARK); _style_ax(ax2)
ax2.plot(fpr, tpr, color=C_BLUE, lw=2.5, label=f'AUC = {auc:.3f}')
ax2.plot([0,1],[0,1], '--', color='#6e7681', lw=1.5, label='Random')
ax2.fill_between(fpr, tpr, alpha=0.15, color=C_BLUE)
ax2.set_xlabel('False Positive Rate')
ax2.set_ylabel('True Positive Rate')
ax2.set_title('ROC Curve', fontweight='bold', color=C_TITLE)
ax2.legend(facecolor=BG_PANEL, labelcolor=C_TITLE)
fig2.savefig(os.path.join(OUTPUT_DIR, "ecommerce_roc_curve.png"),
            dpi=150, bbox_inches='tight', facecolor=BG_DARK)
plt.close(fig2)
print("  Saved: ecommerce_roc_curve.png")

return model_lr, model_log

if __name__ == "__main__":
    print("=" * 55)
    print("  E-COMMERCE SALES ANALYSIS - Lab 10")
    print("  GUC · CCE · Data Analysis with Python")
    print("=" * 55)
    df = load_and_prepare()
    run_eda(df)
    run_models(df)

```



```
print("\n" + "=" * 55)
print("  FINAL REPORT SUMMARY")
print("=" * 55)
print(f"  Total Orders      : {len(df):,}")
print(f"  Total Sales        : ${df['Sales'].sum():,.0f}")
print(f"  Total Profit       : ${df['Profit'].sum():,.0f}")
print(f"  Profitable Orders  : {df['is_profitable'].mean():.1%}")
print(f"  Avg Profit Margin  : {df['Profit_Margin'].mean():.1f}%")
print(f"  Top Category       : {df.groupby('Category')['Sales'].sum().idxmax()}")
print(f"  Top Region         : {df.groupby('Region')['Profit'].sum().idxmax()}")
print(f"  Top Segment        : {df.groupby('Segment')['Profit'].sum().idxmax()}")
print("=" * 55)
print(f"\n✓ All outputs saved to: {OUTPUT_DIR}")
```